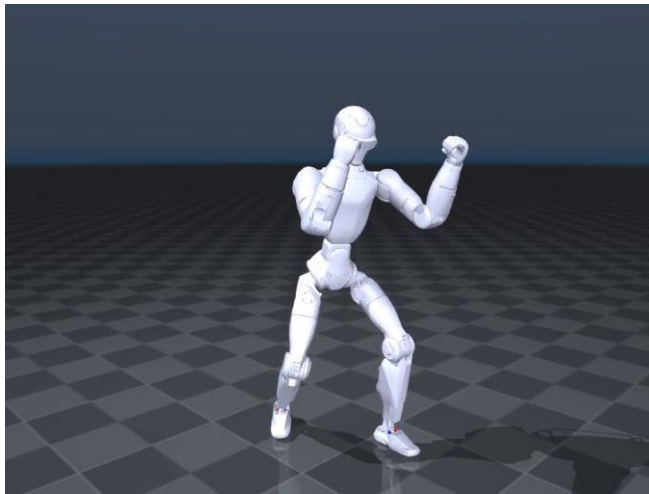


Embodied AI

Lab Manual for GMR

Lab 4 — Make the Robot Copy a Human

September 2026



Shaoqin Li

Applied and Embodied AI Lab
Electrical Engineering Department
The City College of New York

install GMR at home · bring the READY verdict · no GPU

Contents

The shape of this lab	3
What you will do	3
What you should be able to say afterwards	3
Part 0 — Are you set up?	4
Part 1 — What retargeting is	4
Part 2 — Run it	5
Two things in that one line of output	6
Why <code>--start 600 --end 1800</code>	6
Part 3 — Watch it	6
Part 4 — What a motion actually is	6
Part 5 — Measure it	7
Read it carefully — two of the four checks pass	11
Now go and get the wrong answer on purpose	11
Part 6 — Half an architecture	13
Part 7 — Your own motion (<i>homework</i>)	13
Troubleshooting	14
What to hand in	14
Appendix — Instructor Notes	15

Hardware: any laptop, no GPU · **Before you start:** the setup guide, **at home, days earlier**. GMR is a 3.6 GB clone and there is no time to install it during the session. Bring the **READY** verdict from `check_setup.py`. **Also finish Labs 1–3** — Part 6 only means something if you have trained a policy yourself.

Copy the code from the `.md` version of this manual, not from the PDF. Open

`LAB4_video_to_robot_motion.md` in VS Code beside this document and copy from there.

Every **Expected** block below was produced by actually running the command, on a GMR clone with the R1 installed by following the setup guide. Retargeting is deterministic, so **your numbers should match these exactly** — including the ones from your own retargeting run, not just from the file we ship. **If they differ, that is a finding.** Verified 2026-09-01 on Ubuntu 24.04 under WSL2, MuJoCo 3.6.0, Python 3.10.

The shape of this lab

Labs 1–3 all asked the same kind of question: *does it stay up?* This one asks a different one.

You will take **a real human motion capture**, convert it onto the R1, and watch a robot throw punches. It looks superb. Then you measure it, and it turns out **it could not physically have happened** — and that gap is the entire point of the lab.

What you will do

Part		Time
0	Check the setup you did at home	5 min
1	What retargeting is, and what had to be added for the R1	5 min
2	Run it on a boxing capture	10 min
3	Watch it — and be impressed	10 min
4	Open the file and see what a motion actually is	5 min
5	Measure it — the robot goes through the floor	15 min
6	Why this is half an architecture, and Lab 2 is the other half	5 min
7	Your own motion — homework	—

What you should be able to say afterwards

1. What retargeting solves, in one sentence — and what it is never asked about.
2. Why a motion that looks perfect can be physically impossible.
3. **Why "retarget, then RL" is the standard architecture**, and which half you built in Lab 2.
4. How you would catch a bad motion **before** you spent a week training a policy to track it.

Part 0 — Are you set up?

```
conda activate gmr
cd ~/GMR
python ~/r1_lab/exp4/check_setup.py
```

Expected — the last lines:

```
[ok] R1 robot model           r1_scene.xml
[ok] R1 motion mappings      smplx, bvh_xsens
[ok] the boxing capture      2949 KB
[ok] mujoco_menagerie submodule
[ok] video writer (imageio)

READY.  Nothing else to install.
```

✗ If it says **NOT READY**, work through what it lists. **Do not start Part 2 without this.**

You are not blocked if the install defeated you. Ask for `lab4_fallback.tar.gz` — it carries a finished video and the matching motion file, so Parts 3–6, including the measurement that is the whole lesson, work without ever running GMR. Only Part 2 needs the install.

One warning prints on every single command: `xrbotoolkit_sdk not found, skip for now`. It is a VR streaming dependency nothing here uses. Ignore it. It is not an error.

Part 1 — What retargeting is

A motion capture records a **human**: where a person's hips, knees, hands and head were, hundreds of times a second. The R1 is not a person. Different limb lengths, different joint axes, twenty-four motors where a body has hundreds of degrees of freedom.

Retargeting is the translation. For each frame it asks: *what joint angles put this robot's body closest to where the human's body was?* That is an inverse-kinematics problem, solved once per frame.

GMR — General Motion Retargeting — is the research tool that does it. In the setup guide you saw that stock GMR answers `False` for the R1 in every format, and you installed the four things that change that. Confirm it stuck:

```
python - <<'EOF'
from general_motion_retargeting import IK_CONFIG_DICT
for fmt, robots in IK_CONFIG_DICT.items():
    print("  %-12s %s" % (fmt, "R1 OK" if "unitree_r1" in robots else "-"))
EOF
```

Expected:

```
splx      R1 OK
bvh_lafan1 -
bvh_nokov -
bvh_xsens R1 OK
fbx       -
fbx_offline -
xrobot    -
xsens_mvn -
```

Two formats work. **SMPL-X and Xsens BVH. LAFAN1 does not** — and LAFAN1 is the dataset most GMR tutorials use, so if you follow a tutorial you will hit a refusal. That refusal is correct: **no LAFAN1 → R1 mapping has been written yet**. Writing one is real, useful, unclaimed work.

☒ Checkpoint: two formats say **R1 OK**, and you can say what an IK config is.

Part 2 — Run it

This part runs **from the GMR repository**, not from your lab folder:

```
cd ~/GMR
```

GMR ships exactly one motion capture, and it is the one we use: 4249 frames at 120 Hz — 35 seconds of boxing.

```
python scripts/xsens_bvh_to_robot_headless.py \
  --bvh_file assets/xsens_bvh_test/251021_04_boxing_120Hz_cm_3DsMax.bvh \
  --robot unitree_r1 --start 600 --end 1800 --save_path out/mine.pkl
```

Expected — the last two lines:

```
[headless] IK solve rate: 197.2 FPS (6.1s for 1200 frames), motion is 120 fps -> real-time
capable
[headless] motion saved to out/mine.pkl
```

Two things in that one line of output

197 FPS against a 120 fps motion. The solver is running faster than the motion plays. That means retargeting is not an offline batch process — it could run **live**, off a streaming mocap suit, with margin to spare. (Your exact rate depends on what else your laptop is doing; anything above 120 makes the same point.)

Pass `--video_path` and it collapses to about 4 FPS. Fifty times slower. The bottleneck is not the mathematics, it is drawing pixels. Worth remembering the next time something feels slow: measure before you optimise the part you assume is expensive.

Why `--start 600 --end 1800`

Because frames 0–360 are **not the capture**. Every Xsens session opens with the performer standing still in a calibration T-pose. You will meet that trap deliberately in Part 5.

☒ **Checkpoint:** `out/mine.pkl` exists and your solve rate is above 100 FPS.

Part 3 — Watch it

```
python scripts/vis_robot_motion.py --robot unitree_r1 --motion_file out/mine.pkl
```

That needs a working MuJoCo window, so under WSL2 it will fail the same way Lab 1 Part 9b did. If it does, play the copy from the fallback download instead: `reference/r1_boxing.mp4`, ten seconds at 960×720. It is the same motion you just produced.

Watch it two or three times, and commit to a judgement before Part 5: does that look like a real robot boxing?

It is genuinely convincing. The guard comes up, the weight shifts, the punches extend and retract, the feet stay planted under the body. Nothing about it looks wrong.

Write down your verdict now. Part 5 is worth far more if you have already decided.

Part 4 — What a motion actually is

Before measuring it, look at what came out:

```
python - <<'EOF'
import pickle, numpy as np
d = pickle.load(open("out/mine.pkl", "rb"))
for k, v in d.items():
    print("%-12s %-14s %s" % (k, str(np.asarray(v).shape), np.asarray(v).dtype))
EOF
```

Expected:

```
root_pos      (1200, 3)      float64
root_rot      (1200, 4)      float64
dof_pos       (1200, 24)     float64
fps           ()           int64
robot         ()           <U10
```

That is the whole thing. For each of 1200 frames:

<code>root_pos</code>	where the pelvis is — 3 numbers, metres
<code>root_rot</code>	which way the pelvis faces — 4 numbers, a quaternion
<code>dof_pos</code>	the angle of each of the 24 joints, radians

Notice what is not there. No velocities. No torques. No contact forces. No mass. **Nothing about whether any of this is physically possible** — it is a list of poses and the times to hold them.

That absence is the finding you are about to measure.

Part 5 — Measure it

Create `exp4_check_motion.py` in `~/r1_lab/exp4`:

```
"""Experiment 4 -- is this motion physically possible?

The retargeted motion LOOKS right. This asks whether a real robot could have
produced it. Run it on your own output:

    python exp4_check_motion.py reference/r1_boxing.pkl
    python exp4_check_motion.py out/my_motion.pkl --robot unitree_r1

Needs the `gmr` conda environment (it reads the robot model out of GMR).

WHAT IT CHECKS
1. FEET      do they ever touch the ground
2. PELVIS    does the body height move at all
3. LIMITS    is any joint driven past its mechanical stop
4. SPEED     does any joint move faster than a motor could
```

```

A motion can pass every visual inspection and fail all four. That is the
point of the experiment.
"""
import argparse, os, pickle, sys
import numpy as np
import mujoco

try:
    from general_motion_retargeting import ROBOT_XML_DICT
except ImportError:
    sys.exit("Run this inside the `gmr` environment: conda activate gmr")

# ----- EDIT ZONE
FLOAT_TOL_CM = 1.0      # closer than this to the floor counts as "touching"
STILL_TOL_CM = 0.5      # pelvis range below this counts as "frozen"
MAX_JOINT_SPEED = 20.0  # rad/s -- generous for a real servo
# -----

def load(path):
    d = pickle.load(open(path, "rb"))
    need = ("root_pos", "root_rot", "dof_pos")
    if not all(k in d for k in need):
        sys.exit(f"{path} is missing {need}; is it a GMR motion pickle?")
    return (np.asarray(d["root_pos"]), np.asarray(d["root_rot"]),
            np.asarray(d["dof_pos"]), float(d.get("fps", 30)))

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("motion")
    ap.add_argument("--robot", default="unitree_r1")
    a = ap.parse_args()

    rp, rr, dp, fps = load(a.motion)
    m = mujoco.MjModel.from_xml_path(str(ROBOT_XML_DICT[a.robot]))
    d = mujoco.MjData(m)
    n = len(rp)

    print(f"\n{' '*64}\n {os.path.basename(a.motion)} -> {a.robot}")
    print(f" {n} frames @ {fps:g} fps = {n/fps:.1f} s | {dp.shape[1]} joints")
    print(f"{'=' * 64}")

    # the spheres and capsules that would actually hit the floor
    foot = [g for g in range(m.ngeom)
             if (mujoco.mj_id2name(m, mujoco.mjtObj.mjOBJ_GEOM, g) or "").endswith("_collision")]
    if not foot:
        foot = list(range(m.ngeom))

    low = np.empty(n)
    for i in range(n):
        d.qpos[3:], d.qpos[3:7], d.qpos[7:] = rp[i], rr[i], dp[i]
        mujoco.mj_forward(m, d)
        # bottom of each contact primitive, not its centre

```



```

low[i] = min(d.geom_xpos[g, 2] - m.geom_size[g, 0] for g in foot)

# ----- 1. FEET
print("\n 1. FEET -- does the robot ever touch the ground?")
print(f"      lowest contact point: min {low.min()*100:+6.2f} cm"
      f"      mean {low.mean()*100:+6.2f} cm      max {low.max()*100:+6.2f} cm")
touch = int((np.abs(low) < FLOAT_TOL_CM / 100).sum())
if touch == 0:
    state = "FLOATING the whole time" if low.min() > 0 else "SUNK THROUGH THE FLOOR"
    print(f"      -> {state}. 0 of {n} frames make contact.")
else:
    print(f"      -> touches down on {touch}/{n} frames ({100*touch/n:.0f}%)")

# ----- 2. PELVIS
rng = (rp[:, 2].max() - rp[:, 2].min()) * 100
print("\n 2. PELVIS -- does the body rise and fall like a real one?")
print(f"      height {rp[:,2].min():.3f} .. {rp[:,2].max():.3f} m      (range {rng:.2f} cm)")
verdict = ('FROZEN. A moving human does not hold this constant.'
          if rng < STILL_TOL_CM else 'moves, as a real body would')
print(f"      -> {verdict}")

# ----- 3. LIMITS
jnt = [j for j in range(m.njnt) if m.jnt_type[j] != mujoco.mjtJoint.mjJNT_FREE]
lim = m.jnt_range[jnt]
lim = lim[:dp.shape[1]]
lo, hi = lim[:, 0], lim[:, 1]
over = (dp < lo - 1e-6) | (dp > hi + 1e-6)
print("\n 3. LIMITS -- is any joint pushed past its mechanical stop?")
if over.any():
    worst = np.argsort(-over.sum())[0:3]
    print(f"      {over.any(1).sum()}/{n} frames violate a limit")
    for j in worst:
        if over[:, j].sum():
            nm = mujoco.mj_id2name(m, mujoco.mjtObj.mjOBJ_JOINT, jnt[j]) or f"joint{j}"
            ex = max(abs(dp[:, j].min() - lo[j]), abs(dp[:, j].max() - hi[j]))
            print(f"      {nm:28s} {over[:,j].sum():5d} frames, worst by {ex:.3f} rad")
else:
    print(f"      -> none. every joint stays inside its range.")

# ----- 4. SPEED
vel = np.diff(dp, axis=0) * fps
peak = np.abs(vel).max()
print("\n 4. SPEED -- could a motor actually move this fast?")
print(f"      peak joint speed {peak:.1f} rad/s      (limit set to {MAX_JOINT_SPEED:g})")
fast = int((np.abs(vel) > MAX_JOINT_SPEED).any(1).sum())
print(f"      -> {'exceeded on ' + str(fast) + ' frames' if fast else 'within a plausible servo budget'}")

# ----- IS THIS EVEN THE MOTION?
# Xsens files open with a stationary calibration T-pose. Measure that by
# accident and you will "discover" a frozen pelvis and floating feet that
# belong to the calibration, not to the capture. This cost the author of
# this script an hour, so the check is built in.
static = rng < STILL_TOL_CM and peak < 2.0

```

```

if static:
    print("\n !! WARNING -- THIS LOOKS LIKE A CALIBRATION POSE, NOT A MOTION.")
    print(f"      The pelvis moves {rng:.2f} cm and the fastest joint reaches")
    print(f"      {peak:.1f} rad/s. Real motion does not look like this.")
    print("      Xsens captures begin with a stationary T-pose. Skip past it:")
    print("      --start 600 --end 1800")
    print("      Every number above describes the T-pose. Re-run before")
    print("      drawing any conclusion from them.")

# ----- VERDICT
fails = [t for t, bad in (
    (f"feet penetrate the floor by {abs(low.min())*100:.2f} cm", low.min() < -
    FLOAT_TOL_CM/100),
    ("feet never contact the ground", touch == 0 and low.min() > 0),
    ("pelvis height is frozen", rng < STILL_TOL_CM),
    ("joint limits violated", bool(over.any()))),
    (f"joint speed implausible on {fast} frames", fast > 0)) if bad]
print("\n" + "=" * 64)
if static:
    print(" VERDICT: withheld -- you measured a calibration pose.")
    print(" Re-run with --start 600 --end 1800 and read the result then.")
    print("=" * 64 + "\n"); return
if fails:
    print(" VERDICT: this motion is NOT physically realisable.")
    for f in fails:
        print(f"      - {f}")
    print("""
This is the lesson of Experiment 4. Retargeting solves ONE problem: put the
robot's hands and feet where the human's were. It succeeds at that, which is
why the video looks convincing. Nothing in the pipeline ever asked about
gravity, contact, torque, or whether the robot could stay upright -- so
nothing in the output respects them.

A retargeted motion is a REFERENCE, not a controller. Making it real is the
job of the policy you trained in Lab 2: track this trajectory while actually
balancing. That two-stage design -- retarget, then RL -- is the architecture
behind every paper in the reading packet.""")
    else:
        print(" VERDICT: nothing implausible found.")
        print("=" * 64 + "\n")

if __name__ == "__main__":
    main()

```

It asks four questions of any motion file: do the feet ever touch the ground, does the pelvis rise and fall, is any joint driven past its stop, and could a motor move that fast.

```
python ~/r1_lab/exp4/exp4_check_motion.py out/mine.pkl
```

Expected:

```
1. FEET -- does the robot ever touch the ground?
   lowest contact point: min -1.15 cm  mean +0.50 cm  max +1.80 cm
   -> touches down on 762/1200 frames (64%)

2. PELVIS -- does the body rise and fall like a real one?
   height 0.648 .. 0.733 m  (range 8.49 cm)
   -> moves, as a real body would

3. LIMITS -- is any joint pushed past its mechanical stop?
   -> none. every joint stays inside its range.

4. SPEED -- could a motor actually move this fast?
   peak joint speed 27.3 rad/s  (limit set to 20)
   -> exceeded on 5 frames

VERDICT: this motion is NOT physically realisable.
- feet penetrate the floor by 1.15 cm
- joint speed implausible on 5 frames
```

Read it carefully — two of the four checks pass

The pelvis moves 8.5 cm, like a real body. No joint is driven past its mechanical stop. Whoever built this did not do it carelessly.

But **the foot goes 1.15 cm below the floor**, and on five frames a joint is asked to move at **27.3 rad/s** — well past what the servo could deliver.

Why it happens, and why it is not a bug

Retargeting was given exactly one problem: **put the robot's limbs where the human's were**. It solves that, well — which is precisely why the video is convincing.

Nothing in the pipeline ever asked about gravity. Or contact. Or torque. Or whether the robot could stay upright. **So nothing in the output respects them**. The floor is not part of the problem it was given, so the foot goes through it.

A tool that solves the problem you gave it is not broken when it fails a problem you never mentioned.

Now go and get the wrong answer on purpose

This is the most useful five minutes in the lab. Retarget the **beginning** of the capture — the part Part 2 told you to skip:

```
python scripts/xsens_bvh_to_robot_headless.py \
  --bvh_file assets/xsens_bvh_test/251021_04_boxing_120Hz_cm_3DsMax.bvh \
  --robot unitree_r1 --start 0 --end 360 --save_path out/tpose.pkl
```

```
python ~/r1_lab/exp4/exp4_check_motion.py out/tpose.pkl
```

Expected:

```
1. FEET -- does the robot ever touch the ground?
   lowest contact point: min +4.53 cm  mean +4.58 cm  max +4.63 cm
   -> FLOATING the whole time. 0 of 360 frames make contact.

2. PELVIS -- does the body rise and fall like a real one?
   height 0.787 .. 0.787 m  (range 0.06 cm)
   -> FROZEN. A moving human does not hold this constant.

4. SPEED -- could a motor actually move this fast?
   peak joint speed 0.6 rad/s  (limit set to 20)
   -> within a plausible servo budget

!! WARNING -- THIS LOOKS LIKE A CALIBRATION POSE, NOT A MOTION.
   The pelvis moves 0.06 cm and the fastest joint reaches
   0.6 rad/s. Real motion does not look like this.
```

That is a far more dramatic result than the real one. *The robot never touches the ground and its pelvis never moves* — a much better story than "the foot sinks a centimetre."

It is also completely wrong. Every number is true of the three-second T-pose and false of the boxing. The giveaway is check 4: **0.6 rad/s is impossibly slow for someone throwing punches.** An implausible number in a check you were not focused on is what exposes a mistake in the one you were.

This exact error was made while building this lab, and it was reported as the headline finding before anybody caught it. The tool now refuses to give a verdict when the input looks like a calibration pose — which is why the warning above exists at all.

A measurement tool that cannot say "I do not trust this input" will eventually hand you a confident wrong answer.

This is the fourth time in the workshop you have met the same shape of problem. Lab 1's NaN check that could not see a blown-up simulator. Lab 2's rising training curve over a policy that could not stand. Lab 3's tool that needed a null control before its results meant anything. And now a set of four measurements that are all correct and all about the wrong three seconds.

☒ **Checkpoint: produce both results, and say what made the second one detectably wrong.**

Part 6 — Half an architecture

So you have a motion that looks perfect and cannot be executed. What is it good for?

It is a reference, not a controller. It says what the robot should look like. It says nothing about how to stay standing while doing it — and as Labs 1 and 3 showed, staying standing is the hard part.

You already built the other half.

	what it gives you	what it cannot do
Retargeting (this lab)	a rich human-like motion, from cheap video or mocap	respect gravity, contact, or torque
RL policy (Lab 2)	balance, push recovery, real physics	invent an interesting motion to perform

Put them together and each covers the other's gap: **retarget a motion to get the reference, then train a policy to track that reference while actually balancing.** The reward gains one term — *stay close to the reference pose* — on top of everything Lab 2 already had.

That two-stage design is the architecture behind essentially every humanoid-imitation paper in the reading packet. You have now built both halves and measured why neither is sufficient alone.

And notice what this lab gives you for free: a way to reject a bad motion in ten seconds, before spending an hour of training on it. If the reference is physically impossible, the policy will spend its whole budget failing to track something no robot could do.

Part 7 — Your own motion (*homework*)

The R1 accepts **SMPL-X** and **Xsens BVH**. In practice that means **AMASS**, which is free but requires registration — so it cannot happen inside a session.

1. Register at the AMASS site and download any subject's `.npz` motion.
2. Retarget it: `python scripts/smplx_to_robot.py --smplx_file <your file>.npz \ --robot unitree_r1 --save_path out/yours.pkl`
3. **Measure it before you admire it:** `python ~/r1_lab/exp4/exp4_check_motion.py out/yours.pkl`
4. Report what failed. Dance and ground-lying motions are the known-hard cases; `TEST_MOTIONS.md` in the repo lists which public clips retarget cleanly.

LAFAN1 will not work — there is no R1 config for it. Writing `bvh_lafan1_to_r1.json` would open up a large public dataset, and it is genuinely unclaimed work if you want it.

`exp4_check_motion.py` has an **EDIT ZONE** at the top: the floor tolerance, the "frozen pelvis" threshold, and the 20 rad/s speed limit. That last one is a guess about a servo, not a measurement. Look up the R1's real joint speed limit and put it in — the verdict on the boxing motion may change, and if it does, that is your finding.

Troubleshooting

Symptom	Cause & fix
<code>ModuleNotFoundError: general_motion_retargeting</code>	you forgot <code>conda activate gmr</code> — by far the most common
<code>ModuleNotFoundError: mink</code>	<code>pip install -e .</code> not run, or run in the wrong environment
<code>unitree_r1</code> rejected by <code>--robot</code>	you are on a LAFAN1 script. Use <code>xsens_bvh_to_robot_headless.py</code>
every format prints <code>-</code> in Part 1	the setup guide's installer did not run. Run it again
<code>xrobotoolkit_sdk not found</code>	not an error. Ignore it, it prints every time
<code>gladLoadGL error</code>	no display — use the <code>_headless</code> script, or play the fallback video
Everything is ~50× slower	you passed <code>--video_path</code> . Rendering is the bottleneck, not the IK
<code>VERDICT: withheld</code>	you measured the T-pose. <code>--start 600 --end 1800</code>
Numbers differ from this manual	tell the instructor — retargeting is deterministic, they should not

What to hand in

1. Your `check_setup.py` verdict.
2. Your solve rate from Part 2, and what it implies about running retargeting live.
3. **Your written judgement from Part 3, made before you measured anything.**
4. The four measurements from Part 5, and the verdict.
5. One paragraph: **why does a tool that produces an impossible motion count as working correctly?**
6. The T-pose result, plus one sentence on **which number gave the mistake away, and why.**
7. One sentence: **what does Lab 2 supply that this lab cannot, and vice versa?**

Appendix — Instructor Notes

- **Setup must be done days early, and verified.** This is the only lab with a 3.6 GB download and a second conda environment. Collect the `check_setup.py` verdicts before the session; there is no recovering a failed install in the room.
- **Stock GMR has no R1.** The setup guide makes students prove that themselves (every format `False`) before installing the four pieces that fix it. That is the honest version of "this project added R1 support to GMR", and it takes five minutes.
- **Nobody is blocked by a failed install.** `lab4_fallback.tar.gz` carries Parts 3–6. Say so up front so a student who lost the install fight does not sit out the lesson.
- **Part 3 is not filler.** The measurement in Part 5 only lands against a stated judgement. Make them commit out loud, or in writing.
- **Part 5's second half is the lab.** Getting the dramatic wrong answer on purpose, and then finding which number betrayed it, is the transferable skill. Budget the time for it.
- **Tell them it was a real mistake,** made here, reported as a finding before it was caught. It is more instructive than any invented example, and it is the reason the tool now withholds verdicts.
- **The LAFAN1 gap is a genuine open task.** A student who wants a project can write that IK config.
- **Timing:** Parts 0–2 about 20 minutes, Part 3 10, Parts 4–5 20, Part 6 5. The retargeting itself takes under 10 seconds, so the time is discussion, not compute.